

Mojo for RISC OS - the compiler, rosc and the ex...

Mojo for RISC OS

How a fork of the Mojo compiler, a small Rust linker called `rosc`, and a C runtime together put Mojo programs on RISC OS 5 — first on an emulated StrongARM, then on the emulated Raspberry Pi 4 — and what the Wimp demos that run there teach about every layer underneath.

Porting a language to RISC OS normally means writing a RISC OS code generator. This project took another route. The compiler emits ordinary ARM object files, and a separate back end turns them into RISC OS programs. `rosc`'s README puts the idea in one line:

`rosc` is **not a compiler**. It is the part that knows what RISC OS is: image formats, the application slot, the AIF header, filetypes.

So the port became a runtime and a linker, and not a new code generator. By the evening of 12 September 2026, Mojo programs were printing to the screen, checking their own arithmetic, and running as desktop applications — Othello and a zoomable Mandelbrot set — on the emulated Pi 4.

These documents

Chapter	What it covers
1. A language port as a runtime and a back end	the repositories, the two profiles, the chain
2. From .mojo to an ARM object	the fork, the ARM backend, and a NEON fault
3. rosccl: ELF in, AIF out	a code walk of the linker
4. rostrt: the smallest runtime	start-up, memory, output, what is missing
5. The PRM as data	generating a library for 440 SWIs

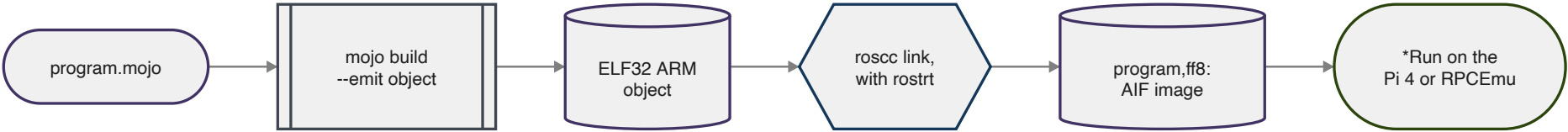
These documents, continued

Chapter	What it covers
6. A Wimp application: Othello	a desktop task in Mojo, and the bugs it found
7. Fixed point on the desktop: Mandelbrot	Q16.16, run-length redraw, and a zoom floor
8. Running and verifying it	the Pi 4 farm, typing over QMP, what ran
9. The unpublished half	modules, reproducibility, and an honest ledger

At a glance

Compiler	a fork of Mojo, emitting ELF32 ARM objects for armv8a-none-eabi or armv4-none-eabi
Back end	rosc: 977 lines of Rust; ELF in, AIF executable out, at &8000
Runtime	rostrt: C and assembly, built with clang, one C shim per SWI
Library	439 generated bindings covering 440 of 584 documented SWIs
Targets	Cortex-A72 on the QEMU Pi 4; StrongARM on RPCEmu
Images	hello world 9,824 bytes; Othello 13,852; Mandelbrot 12,232

The chain, end to end



This document describes the published state on 13 September 2026: MojoRISCOS branch `riscos` at `3ce3feed0d`, ROSCC at `707a35f`, and the QEMU fork at `e7e6deffae`. Where part of the toolchain used for the runs is not in those repositories, the text says so.

1. A language port as a runtime and a back end

This chapter sets out the shape of the work before the detail: which repository holds which piece, the two RISC OS machines the output targets, and the decision that sent Pi 4 testing to the QEMU fork. It ends with the full build chain, with every command and flag the published scripts use.

Five repositories, one toolchain

Repository	What it holds	Licence
MojoRISCOS	the Mojo fork: RISC OS documents, the generated <code>riscos/</code> bindings, three demos and a build script	Apache-2.0 with LLVM exceptions
ROSCC	<code>rosc</code> , the <code>rostrt</code> runtime, and the binding generator	MIT
RISCOSQEMUA72	the emulated Pi 4 the A72 programs run on, its four-machine farm, and the typing tool	GPL-2.0-or-later

Five repositories, continued

Repository	What it holds	Licence
RPCEMU_INSTRUMENT	the StrongARM sandbox: an instrumented RPCEmu with a fault trap and a guest portal	GPL-2.0-or-later
ROSASM	a RISC OS assembler and disassembler; a sibling that names <code>rosc</code> as its C-compiler counterpart	—

Why a back end, not a code generator

The design turns a language port into two smaller problems. The ROSCC README argues it directly. Porting a language to RISC OS normally means writing a RISC OS code generator. With `rosc`, it means **writing a runtime**: teach the language how to reach the OS, then hand `rosc` ordinary ARM objects.

That is already true of the tree, not just an aspiration. Every image `rosc` produces links objects from **two independent toolchains**. The runtime and the SWI shims are C and assembly compiled by clang, and the application code comes from the Mojo compiler. Nothing in `rosc` knows which produced a given object. The README lists GCC, Rust, Zig and FreePascal as candidates, without touching their back ends.

What rosc knows, and what it does not

rosc knows	rosc does not know
the AIF executable header, loaded at &8000	which compiler produced an object
the application slot, and where a heap can go	Mojo's types, its runtime, or its standard library
ARM ELF relocations, and how to apply them	how to generate code
RISC OS filetypes, through the , ff8 suffix	anything about SWIs; the runtime issues them

Two profiles

RISC OS 5 runs on ARM machines from very different eras, and the port targets two:

Profile	CPU	Runs on	Code constraints
riscos-sa	StrongARM SA-110, ARMv4	RPCEmu, an emulated Risc PC	no hardware divide, no movw/movt, no unaligned loads and stores, ARM state only
riscos-a72	Cortex-A72, ARMv8-A in AArch32	the emulated Raspberry Pi 4	the modern instruction set; FP registers disabled in practice (chapter 2)

The model is the classic one for RISC OS: compile for ARMv4 to run anywhere, and for the A72 when the Pi has to work for a living. One runtime source serves both, and `tools/build-rt.sh` builds it twice.

The platform decision

A review of `rosc` on 10 September recorded one decision that shaped everything after it: **the Pi 4 profile is tested on the QEMU fork, not on RPCEmu**. A plan to teach RPCEmu's interpreter the A72 instruction set was dropped. RPCEmu stays the StrongARM sandbox.

That decision is why the [QEMU A72 walkthrough](#) begins with a compiler: its first chapter explains that the emulated Pi 4 exists because `rosc`'s output needs an ARMv8 core to run on.

The chain, as it ran

On 12 September the published build script, `riscos-test/pi4.sh`, took a Mojo program to the Pi 4 in four steps. The exact commands follow; host-specific install paths are described rather than reproduced.

1. Compile with the fork's `mojo`, built locally with LLVM's ARM backend enabled:

```
"$MOJO" build --emit object \  
  --target-triple $TRIPLE --target-cpu $CPU \  
  --target-features +strict-align,-fpregs \  
  -I "$ROOT/mojo/stdlib" -I "$ROOT" -I "$ROOT/riscos-test" \  
  -o "build/$name.o" "$src" 2>"build/$name.log"
```

with `TRIPLE=armv8a-none-eabi` and `CPU=cortex-a72`.

2. Link with `rosc`, naming the runtime objects explicitly:

```
"$ROSCC" link --entry _start -o "$out" \  
  "$RT/crt0-$TAG.o" "build/$name.o" \  
  "$RT/rostrt-$TAG.o" "$RT/wimp-$TAG.o" \  
  "$RT/swis_os-$TAG.o" "$RT/swis_wimp-$TAG.o" \  
  "$RT/aeabi-$TAG.o" "$RT/atomics-$TAG.o"
```

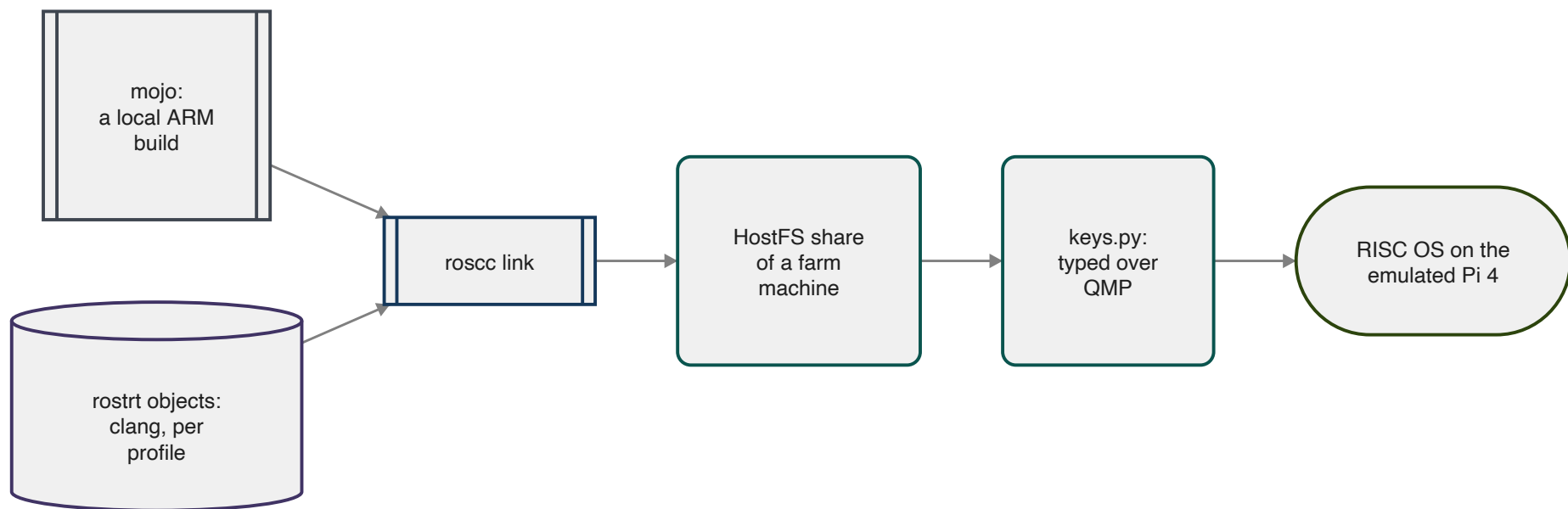
The result is an AIF executable named with a `.ff8` suffix, which carries its RISC OS filetype across a host file system, plus an `.elf` sidecar for disassembly.

3. Deliver by copying the image into a farm machine's HostFS share.

4. Run by typing into the guest over QMP:

```
keys.py alpha --taskwindow --line "Run HostFS:$.hello_world,ff8"
```

The chain, with its tools



Published, and not

Three pieces of what ran on 12 September are not in the published repositories. The rest of this document says so each time it matters, and chapter 9 gathers them:

- **The ARM-enabled compiler.** The fork's published build configuration does not enable LLVM's ARM backend. The `mojo` used was a local build.
- **rosc's relocatable-module writer.** The ROSC README advertises `rosc link --module`; the published source has no such option.
- **The console demos.** Both Wimp demos import helpers from a `demos/` directory that is not tracked, so a clone can build only `hello_world`.

2. From .mojo to an ARM object

The first half of the chain is the compiler. This chapter covers the fork's history, what it actually changes, how the compiler was made to emit 32-bit ARM, the two target flags the Pi 4 build cannot do without, and the route the project took before the compiler could emit ARM directly.

The fork's history

The MojoRISCOS branch `riscos` holds 53,832 commits, but almost all of them are upstream Mojo. The history has four parts:

Part	Commits	What it is
upstream Mojo, to base f66d4d522c (18 August 2026)	53,617	the compiler, standard library and tooling as published upstream
a Windows host port, on a separate root	201	the Mojo compiler built and run on Windows x64, which is where the RISC OS work began
the merge, eec94331fb (8 September)	1	adopts the Windows port's tree, and introduces RISCOS-PORT.md
after the merge	13	3 housekeeping commits and 10 marked <code>[riscos]</code>

The three housekeeping commits separate the fork from its upstream. They stop it presenting as Modular's repository, add a `NOTICE` and a licensing safety rail in `.gitignore`, and remove 1,149 files of Modular's documentation.

The ten RISC OS commits

Commit	When	What
b5c9ec59db	11 Sep 09:16	record whole-OS shim generation, and what is still unbound
f325531965	11 Sep 09:46	track the generated bindings: riscos/, 48 files
9db30e1167	12 Sep 20:29	hello world through the OS library, built and run on the Pi 4
a720f9601a	12 Sep 20:59	stop the Pi 4 target reaching for NEON
7a3b81b82d	12 Sep 20:59	Othello as a Wimp application, and the window support it needed

The ten RISC OS commits, continued

Commit	When	What
d41626b038	12 Sep 21:15	build the console demos for the Pi 4, and run them there
805c4551eb	12 Sep 21:25	Wimp_Initialise: allocate the out-cell in a frame that survives
6e23dc2431	12 Sep 21:25	Othello: the score clear of the board, and legal moves shown
c4b9c847dd	12 Sep 21:44	the Mandelbrot set in a window, in Q16.16
3ce3feed0d	12 Sep 22:15	Mandelbrot: Select zooms in, Adjust zooms out

The branch was force-pushed with this full history after the last commit. An earlier published snapshot, a single flattened commit without the `max/` directory, is no longer part of it.

What the fork changes, and what it does not

Reading the branch against its upstream base gives a clear answer. Every RISC OS change in the tracked tree is in one of these places:

Area	Change
documents	RISCOS-PORT.md, the dated port log; README.md rewritten for RISC OS
identity and licensing	NOTICE, CONTRIBUTING.md, and the .gitignore rail
bindings	riscos/: 45 generated modules, 2 hand-written appendices, and an __init__
demos	riscos-test/: hello_world, othello_wimp, mandelbrot_wimp, and pi4.sh

The compiler itself (`KGEN/`), the build configuration (`bazel/`) and the standard library (`mojo/stdlib`) contain **no RISC OS change at all**. The OS-facing bottom layer that Mojo's standard library needs is C, in ROSCC's runtime, and not Mojo code. Chapter 4 walks through it.

Teaching the compiler to emit ARM

Mojo's compiler already treats 32-bit ARM as a known target, so no C++ change was needed to emit AArch32. What was missing was the ARM code generator in LLVM itself. The fork's build fetches LLVM through Bazel, and the backends it compiles are a list in the build configuration. The port log's reconnaissance, dated 8 September, found the list and concluded that adding `"ARM"` was the entire change.

That is gate **R1**, recorded as done the same day:

- the compiler was rebuilt with the ARM backend: **14,512 build actions, about an hour**
- `--emit asm` and `--emit object` produced correct AArch32 for both profiles, straight from a `.mojo` file: `.cpu cortex-a72` for the Pi 4, `.cpu strongarm110` for RPCEmu
- the first native image, `hellomojo.ff8`, was **432 bytes** of AIF, with its start-up code, `main` and `OS_Exit` confirmed by disassembly

The published build configuration does not contain that change. Its LLVM backend list is `AArch64`, `NVPTX`, `RISCV`, `SPIRV` and `X86`. The build has a hook for extra targets, and the module file does not use it. The `mojo` that built every program in this document was a local build.

INFERRED: a clean clone would build a compiler that refuses `armv8a-none-eabi`, with the *no available targets* error the port log records from before R1.

No RISC OS target in the driver

The compiler driver does not know a `riscos` target. That is gate **R2**, still open. The build uses the driver's stock options instead:

Option	Value for the Pi 4
<code>--emit</code>	object: stop at an ELF object
<code>--target-triple</code>	<code>armv8a-none-eabi</code>
<code>--target-cpu</code>	<code>cortex-a72</code>
<code>--target-features</code>	<code>+strict-align,-fpregs</code>

The link is a separate, explicit `rosc` step, because the driver links with the host's own linker, `link.exe` on Windows and `cc` elsewhere. Folding `rosc` into the driver's platform configuration is what R2 would do.

+strict-align

A StrongARM does not fault on a misaligned load or store; it **rotates** the value. So code for these machines must never make an unaligned access, and the flag tells LLVM so. It has been on the Pi 4 build line from its first commit, `9db30e1167`, as well as in the StrongARM profile's constraints. Chapter 4 tells the story of a string that came back with a hole in it because the runtime ignored an alignment request.

-fpregs, and the fault that made it compulsory

The build script's comment is emphatic: the flag *is not optional*. Asked for a Cortex-A72, LLVM assumes the A72's NEON unit is available, and uses it.

Commit `a720f9601a` records what happened. In a Wimp program, LLVM vectorised an ordinary stack clear into two NEON instructions: a `vmov.i32 d0, #0` and a `vst1.64` store. RISC OS had not enabled the floating-point unit for the program. So the program stopped with *Internal error: undefined instruction at &000084F4*. The StrongARM builds had never seen this, because ARMv4 has no NEON to reach for.

NEON instructions, counted

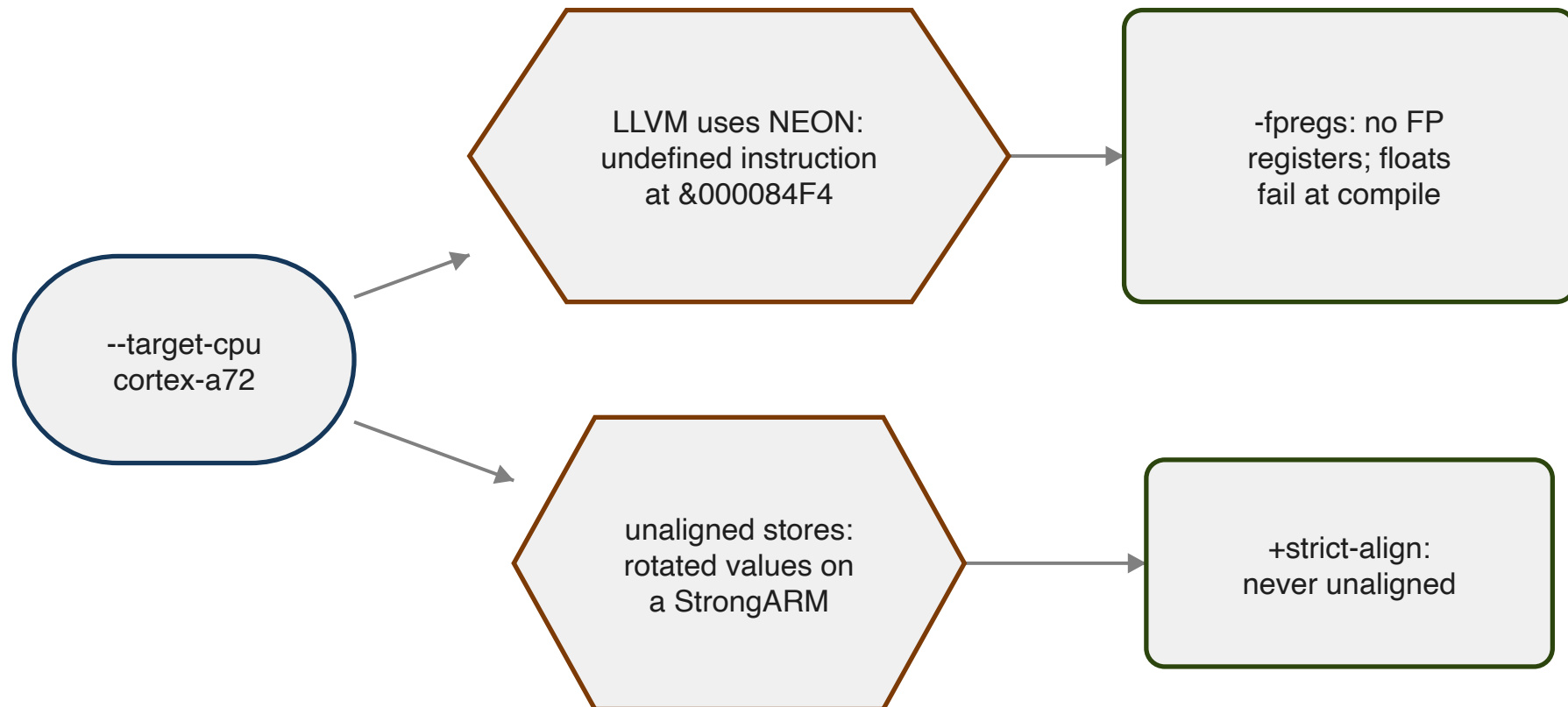
Build	NEON-space instructions in the program
cortex-a72	17
cortex-a72 with <code>-fpregs</code>	0

`-fpregs` removes the floating-point registers from the target entirely. It was chosen over the narrower `-neon` deliberately: with no FP registers at all, **any use of floating point fails at compile time**, instead of at an address in a running program.

That fits the runtime. `rostrt` has no soft-float library, so a `Float64` anywhere fails at link time anyway. The idiom is fixed point, which chapter 7 shows in action.

INFERRED: enabling the FP unit for a task, through RISC OS's `VFPSupport` module, would let A72 programs use hardware floating point. No code in the runtime creates such a context, and the database of SWIs the bindings come from does not include that module. The fork's profile table still lists VFPv4 and NEON for the A72 profile. That entry is an aspiration, not the current state.

Why the flags matter



The route before R1: `rosc` ingest

Before the compiler could emit ARM itself, `rosc` had a second job, which is still in its source as `rosc ingest`. It took LLVM IR produced by the Windows build of Mojo for its own host, and made an ARM object of it:

1. parse the IR text
2. retarget it to `armv8a-none-eabi`, with that target's data layout
3. **strip the function attributes** naming the host CPU and its features; the comment in the source cites the host's Alder Lake attributes
4. verify the module, and emit a static object through LLVM

The port log records that Mojo's IR retargeted cleanly to Cortex-A72. **INFERRED:** IR generated for a 64-bit host has 64-bit pointer layouts baked in, and swapping the data layout afterwards is not sound in general. That is a good reason for the move to a compiler that emits ARM directly. The ingest route remains as history, alongside a `rosc demo` subcommand that builds a two-function smoke-test object.

3. rosc: ELF in, AIF out

rosc's linker is small enough to read in an afternoon: 977 lines of Rust in three files. LLVM is used only by the older `ingest` and `demo` subcommands; the linker and the image writer are pure Rust. This chapter walks through the linker in the order it works — reading objects, laying out sections, resolving symbols, building a GOT, applying relocations and writing the AIF header — and ends with its known defects. Each defect was checked with a small probe.

File	Lines	Role
src/main.rs	371	subcommands; the link arguments; attaching the runtime
src/elf.rs	243	an ELF32 ARM relocatable-object reader, and an ar archive reader
src/aif.rs	363	the static linker, the AIF writer, and the <code>.elf</code> sidecar

Reading objects

`elf.rs` accepts exactly one kind of file, and says precisely why it rejects anything else:

```
if d.len() < 52 || &d[0..4] != b"\x7fELF" {
    return Err(format!("{path}: not an ELF file"));
}
if d[4] != 1 || d[5] != 1 {
    return Err(format!("{path}: not ELF32 little-endian"));
}
if rd_u16(d, 16) != 1 {
    return Err(format!("{path}: not relocatable (e_type)"));
}
if rd_u16(d, 18) != 40 {
    return Err(format!("{path}: not ARM (e_machine)"));
}
```

What the reader keeps

From an accepted object it keeps three things:

Kept	Rule
symbols	every entry, index-exact; <i>global</i> means STB_GLOBAL binding only
sections	only allocated PROGBITS or NOBITS sections with non-zero size; .bss becomes real zeros; .ARM.exidx, notes and comments are dropped
relocations	REL sections only, and only those targeting kept sections

An input can also be an `ar` archive. The comment explains why: `mojo build` emits an archive for a multi-file program. **Every member is included**; there is no pulling in only the members that satisfy undefined symbols.

Layout

Sections are laid out in two passes over the objects, in command-line order: read-only sections first, then writable ones, starting at `&8080`, just past the header:

```
let mut cursor = BASE + HDR;
for pass in 0..2 {
  for (oi, o) in objs.iter().enumerate() {
    for (si, s) in o.sections.iter().enumerate() {
      if (pass == 0) == elf::is_writable(s) {
        continue;
      }
      cursor = align4(cursor);
      addr.insert((oi, si), cursor);
      cursor += s.data.len() as u32;
    }
  }
}
```

`align4` rounds to a four-byte boundary. It does **not** read each section's own alignment requirement. That is the first known defect: a probe that declared a C global aligned to 16 bytes found it placed at `&933C`, which is not.

Symbols

Global symbols go into a single hash map, from name to address:

```
for (oi, o) in objs.iter().enumerate() {
    for sym in &o.symbols {
        if sym.shndx == 0 || !sym.bind_global {
            continue;
        }
        if let Some(&si) = o.keep.get(&(sym.shndx as usize)) {
            let a = addr[&(oi, si)] + sym.value;
            globals.insert(sym.name.clone(), a);
        }
    }
}
```

Two more defects follow directly from these lines, and both were confirmed by probes:

- **A later definition silently replaces an earlier one.** Linking the same object twice succeeds.
- **Weak symbols are not global.** Only `STB_GLOBAL` counts, so a weak definition in one object called from another fails with `undefined: weak_fn`. The review noted why the demos survive this: Mojo emits `linkonce_odr` functions, and their references happen to stay within one object.

A GOT, for a static link

A static executable normally needs no global offset table. Mojo's output uses one anyway: it reaches its closures through `R_ARM_GOT_PREL` relocations even in a static link. So `rosc` synthesises a GOT. It makes one slot per unique symbol name referenced that way, places the slots after every section, and fills each with the symbol's address.

The code carries a comment about a bug found on 12 September (commit `d2ab069`):

```
// cursor has to move to got_base, not merely past it: the slots are
// placed from the aligned address while cursor was advanced from the
// unaligned one, so the image buffer came out up to 3 bytes short and
// the copy panicked. It only ever aligned by luck – every .bss so far
// had happened to be a multiple of 4.
let got_base = align4(cursor);
cursor = got_base;
```

_end: where the image stops

After the GOT, rosc defines one symbol of its own:

```
// _end: the first address past the image, defined here because only the
// linker knows it. A runtime wanting a heap should claim it from the
// application slot – RISC OS hands a program everything from here up to
// the limit OS_GetEnv returns in R1 – rather than reserving one inside
// the image. Reserving it is what made a hello world 304 KB, of which
// 299,312 bytes were rostrt's heap_area and global_arena written out as
// zeros because .bss is materialised rather than declared.
globals.insert("_end".to_string(), align4(cursor));
```

This one symbol took hello world from 304 KB to under 10 KB. Chapter 4 shows the runtime side.

Relocations

The core of any static linker is applying relocations, and in rosc that is one function. It handles the set a static ARM executable needs:

```
pub fn apply(rtype: u8, word: u32, s: u32, p: u32) -> Result<u32, String> {
```

```

Ok(match rtype {
  // S + A, addend in place (REL form)
  R_ARM_ABS32 => s.wrapping_add(word),
  R_ARM_CALL | R_ARM_JUMP24 => {
    let off = (s as i64 - 8 - p as i64) / 4;
    if !(-0x80_0000..0x80_0000).contains(&off) {
      return Err(format!("branch out of range: {s:#x} from {p:#x}"));
    }
    (word & 0xFF00_0000) | ((off as u32) & 0x00FF_FFFF)
  }
  R_ARM_MOVW_ABS_NC => {
    let imm = s & 0xFFFF;
    (word & 0xFFF0_F000) | ((imm & 0xF000) << 4) | (imm & 0xFFFF)
  }
  R_ARM_MOVT_ABS => {
    let imm = (s >> 16) & 0xFFFF;
    (word & 0xFFF0_F000) | ((imm & 0xF000) << 4) | (imm & 0xFFFF)
  }
  R_ARM_V4BX => word,
  // S + A - P (A = signed in-place addend)
  R_ARM_REL32 => (s as i64 + word as i32 as i64 - p as i64) as u32,
  other => {
    return Err(format!(
      "unsupported relocation type {other} - extend the linker"
    ))
  }
})
}

```

The relocation types

Type	Number	Computes	Used for
R_ARM_ABS32	2	$S + A$	data words and literal pools
R_ARM_REL32	3	$S + A - P$	PC-relative data; added at R1
R_ARM_CALL, JUMP24	28, 29	$(S - 8 - P) / 4$ into 24 bits, range-checked	BL and B

Relocations, continued

Type	Number	Computes	Used for
R_ARM_V4BX	40	nothing	marks a BX for ARMv4
R_ARM_MOVW_ABS_NC, MOVW_ABS	43, 44	low and high 16 bits, packed as imm4:imm12	address loads on the A72
R_ARM_GOT_PREL	96	GOT slot + $A - P$, handled before apply	Mojo's closures

Anything else is a hard error that names its number, and asks for the linker to be extended.

Reading the code shows the addend handling is uneven:

- `ABS32` and `REL32` honour the addend stored in the instruction.
- **Branches discard it**: the in-place addend is masked out. This is recorded as a defect.
- **INFERRED**: the MOVW and MOVW cases also ignore any in-place addend. That does not affect the runtime, whose address loads reference their string symbols directly.

The AIF header

The output is always an **AIF executable**, loaded at `&8000`: 32 words, 128 bytes, then the image. The source writes it field by field:

```
hdr[0] = NOP; // no decompression
hdr[1] = NOP; // not self-relocating
hdr[2] = NOP; // no zero-init code (bss materialised)
hdr[3] = bl(BASE + 0x0c, entry); // BL ImageEntryPoint
hdr[4] = SWI_EXIT; // last-ditch exit instruction
hdr[5] = ro_incl_hdr; // read-only size incl. header
hdr[6] = rw_size; // read-write size
hdr[7] = 0; // debug size
hdr[8] = 0; // zero-init size (materialised in file)
hdr[9] = 0; // debug type
hdr[10] = BASE; // image base
hdr[11] = 0; // workspace
hdr[12] = 32 | 0x100; // 32-bit mode + separate data base
hdr[13] = BASE + ro_incl_hdr; // data base
hdr[16] = NOP; // debug init
```

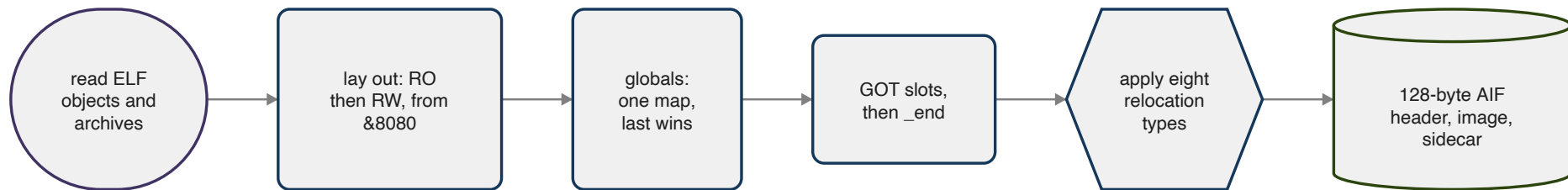
The first three words are no-ops, because the image needs no decompression, no self-relocation and no zero-initialisation code. The fourth is a BL to the entry symbol, which is why every image reports its entry at &8080. The fifth is SWI OS_Exit, in case the entry ever returns. Word 12 declares 32-bit mode and a separate data base.

A probe with a C hello world for StrongARM decoded the header words exactly as written: read-only 4,860 bytes and read-write 4,404, totalling the 9,264-byte file.

The sidecar

Beside every image rosc writes a minimal ELF executable wrapping the same bytes, so `llvm-objdump` can disassemble it. The README describes it as a sidecar *for symbols*. **It carries none:** one program header and one section covering the image.

From objects to an image



Attaching the runtime

`rosc link --rt a72` or `--rt sa` prepends the runtime objects in link order — `crt0`, `rostrt`, `wimp`, `swis_os`, `swis_wimp`, `aeabi`, `atomics` — from a directory found relative to the `rosc` executable. The review noted that this lookup breaks for an installed binary. The Pi 4 build script names the objects itself instead.

Known defects

The ROSCC review of 10 September listed the linker's defects, and each was checked against the published source:

Defect	Checked
section alignment ignored	probe: a 16-byte-aligned global landed at &933C
duplicate definitions accepted; the last wins	probe: the same object linked twice succeeded
weak symbols treated as local	probe: undefined: weak_fn
branch addends discarded	reading the code
malformed input panics instead of reporting an error	reading the code: unchecked slices

Known defects, continued

Defect	Checked
no unit tests	no #[test] anywhere in src/
.bss written out as zeros, not declared as zero-init	header word 8 is 0
no Thumb, no archive member pulling, no section garbage collection	reading the code
the sidecar has no symbols	probe

INFERRED, from reading the code: a local symbol whose name matches a global elsewhere binds to the global, and GOT slots for local symbols of the same name in two objects would collide. The build also produces four compiler warnings, all for unused items.

None of these stopped the demos, which link a small, known set of objects. All of them would matter the moment roscclinks code it has not seen before — which is the whole premise of a back end any compiler can use.

4. rostrt: the smallest runtime

A compiled Mojo program expects things from its environment: a stack, a heap, a way to write bytes, a few entry points in the compiler's runtime library, 64-bit division and atomic operations for reference counts. None of them exist on RISC OS until something provides them. **rostrt** is that something: about 860 hand-written lines of C and assembly, plus the generated SWI shims of chapter 5, compiled freestanding, and making no calls but SWIs. This chapter walks through it, from the first instruction a program executes to what is still missing.

The first instructions

rosc's AIF header branches to `_start`, in `crt0.s`:

```
_start:
    swi    0x10          @ OS_GetEnv: R0 -> env strings, R1 = RAM limit
    mov    sp, r1         @ establish stack at top of our memory slot
    bl     rostrt_init    @ r0 = argc (argv via rostrt_argv_ptr)
    ldr     r1, =rostrt_argv_ptr
    ldr     r1, [r1]
    bl     main
    mov     r2, r0         @ exit code
    mov     r0, #0        @ OS_Exit reason: normal
    swi     0x11
```

The second line fixed the first bug a real machine found. **RISC OS gives an AIF or Absolute image no stack**: the program must set `r13` itself. The first image aborted with `r13 = &80000000`, and the instrumented RPCEmu's fault trap stopped on the faulting instruction. `OS_GetEnv` returns the top of the program's memory in `R1`, which is the natural stack top.

After that, the sequence is:

1. `rostrt_init` splits the command tail into up to 63 arguments, honouring quotes
2. `main(argc, argv)` runs the Mojo program
3. `OS_Exit` ends it

INFERRED: the exit code probably never reaches a caller. `OS_Exit` honours a return code only when R1 holds a particular magic word, and neither `crt0` nor the runtime's `os_exit` sets it.

How the runtime issues a SWI

Every OS call in rostrt is a small C function that binds registers explicitly and issues one `swi` instruction, using clang's register-binding form of inline assembly:

```
__attribute__((noinline)) void os_write0(const char *s)
{
    register const char *r0 __asm("r0") = s;
    __asm__ volatile("swi 0x02" : : "r"(r0) : "r1", "r2", "r3", "r12", "lr", "memory");
}
```

The three lists after the instruction are outputs, inputs and clobbers. Getting those lists exactly right turned out to be the whole game. Chapter 5 shows two bugs that came from getting them wrong.

What Mojo's standard library needs underneath

The fork's standard library is unchanged. Mojo code reaches a handful of C symbols by name, and rostrt supplies each one:

Symbol	What rostrt does
<code>write(fd, buf, n)</code>	one <code>OS_WriteC</code> per byte, with CR inserted before a bare LF; returns the full count
<code>KGEN_CompilerRT_AlignedAlloc, AlignedFree</code>	<code>OS_Heap</code> , over-allocating to honour alignment
<code>KGEN_CompilerRT_GetOrCreateGlobal</code>	a block from a bump arena, remembered in a 256-entry table
<code>SetArgV, PrintStackTraceOnFault, DestroyGlobals</code>	no-ops

What the standard library needs, continued

Symbol	What rostrt does
three <code>AsyncRT</code> CPU-device functions	return the constant 1
<code>KGEN_CompilerRT_fprintf</code>	returns 0
<code>memcpy</code> and friends, <code>__aeabi_mem*</code>	byte loops
64-bit division, in both AEABI and compiler-rt spellings	assembly, in <code>aeabi.s</code>
<code>__atomic_*</code>	SWP loops on StrongARM, LDREX/STREX on the A72

Ten `KGEN_CompilerRT_*` symbols are defined in all. The fork's README still says seven.

print() ends at one symbol

The most useful discovery of the port is recorded in the README: Mojo's `print` bottoms out at exactly one C symbol, `write`. Implement it, and the whole formatting layer above comes for free:

```
long write(int fd, const void *buf, unsigned count)
{
    (void)fd; /* 1=stdout, 2=stderr: same stream on RISC OS for now */
    const unsigned char *p = buf;
    for (unsigned i = 0; i < count; i++) {
        if (p[i] == '\n' && (i == 0 || p[i - 1] != '\r'))
            os_writec('\r');
        os_writec(p[i]);
    }
    return (long)count;
}
```

The CR matters. To the RISC OS VDU drivers, LF means exactly *down one line*. Without a CR, each line starts in whatever column the previous one ended. The source comment describes the symptom: output walks diagonally across the screen, and looks like the program is corrupting its own strings.

The full count must be returned, because the caller asserts on it.

Memory, in three stages

Stage 1: a bump arena

The first runtime, on 8 September, allocated from a static 32 KB arena, and free did nothing. Every intermediate string leaked. Programs that built strings in a loop produced wrong output, which read as a compiler bug.

Stage 2: OS_Heap, and a string with a hole

On 9 September, commit `0797719` moved allocation onto RISC OS's own heap manager, `OS_Heap`: initialise, claim and free. The three calls use the SWI's X form, the only X-form SWIs in the runtime, so a failure returns an error instead of stopping the program.

That commit also found the subtlest bug in the runtime. Mojo's `String` keeps short text inline and moves it to the heap past 24 bytes, and the heap buffer is declared **over-aligned**. `OS_Heap` promises word alignment and nothing more. The allocator ignored the alignment argument. Then a copy into the buffer stored wider than the address allowed — and a StrongARM does not fault on that, it **rotates** the value. So a string of 25 characters came back with a hole in the middle, while its length stayed correct.

The fix honours the alignment by over-allocating, and stashes the raw pointer in the word just below the aligned one, so free knows what to give back:

```
void *KGEN_CompilerRT_AlignedAlloc(u64 alignment, u64 size)
{
    u32 a = (u32)alignment;
    if (a < 8)
        a = 8;
    unsigned char *raw = heap_alloc(size + a + sizeof(void *));
    if (!raw) {
        puts_ro("rostrt: out of heap\n");
        os_exit(1, 0);
    }
    unsigned char *p = raw + sizeof(void *);
    u32 off = (u32)(unsigned long)p & (a - 1);
    if (off)
        p += a - off;
    ((void **)p)[-1] = raw;
    return p;
}
```

The project's own records disagree about whether that fully cured long strings. The same commit's message still calls the hole unfixed, and the Othello demo prints with character output because long strings came back corrupted. The status is recorded as open in chapter 9.

Stage 3: the application slot

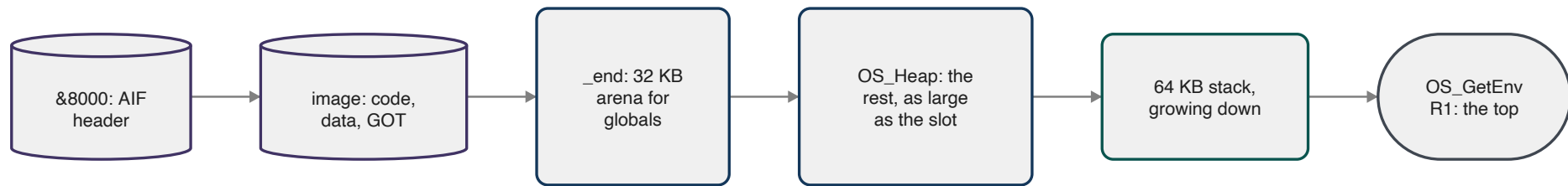
The heap in stage 2 was a quarter-megabyte array in `.bss`. `rosc` writes `.bss` into the image as real zeros, so **a hello world was 304 KB**, of which 299,312 bytes were empty heap. Commit `d2ab069` moved the heap out of the image and into the **application slot**. RISC OS hands a program all the memory between the end of its image and the limit `OS_GetEnv` reports:

```
static void slot_init(void)
{
    if (slot_ready)
        return;
    slot_ready = 1;

#ifdef ROSTRT_STATIC_HEAP
    u32 base = ((u32)_end + 15) & ~15u;
    u32 top = os_mem_limit();
    /* The two grow towards each other: the stack down from the slot top
     * that crt0 put in sp, the heap up from the end of the image. */
    if (top > base + STACK_BYTES + ARENA_BYTES + 16384) {
        top -= STACK_BYTES;
        global_arena = (unsigned char *)base;
        arena_bytes = ARENA_BYTES;
        heap_area = (unsigned char *)(base + ARENA_BYTES);
        heap_bytes = top - (base + ARENA_BYTES);
        return;
    }
    puts_ro("rostrt: application slot too small for a heap\n");
    os_exit(1, 0);
#else
    ...                /* a module: static arrays, claimed from the RMA */
#endif
}
```

`_end` is the symbol `rosc` defines in chapter 3. The limit is asked for again rather than reused. The comment explains why: a Wimp task may have changed its slot size since it started.

The application slot



The result

Image	Before	After
hello_world, ff8	304,248 bytes	9,824
othello_wimp, ff8	308,276 bytes	13,852
mandelbrot_wimp, ff8	306,656 bytes	12,232

The heap also became as large as the slot allows, instead of a fixed 256 KB. Of hello world's 9,824 bytes, Mojo's own code is **227**, as the commit records. **INFERRED** from the sizes of the runtime objects, the rest is the 128-byte header, about 5 KB of runtime code, and `rostrt`'s remaining 4.4 KB of `.bss`: the image is about 97% runtime.

Globals must be the same block every time

`GetOrCreateGlobal` gained a table in the same commit. The comment explains the contract the name implies: ask twice for the same global, and you must get the same block back. Allocating a fresh block each time is invisible in an application, whose `main` asks once. It is quietly fatal in a relocatable module, whose entry points are called again and again. The symptom there was formatted output assembled against a position counter that kept resetting to zero.

INFERRED, and worth verifying with the real compiler: `rostrt`'s signature for this function is a hash, a size, a constructor and a destructor. The upstream compiler runtime's is a name pointer and length, then an initialiser and destructor, and it returns what the initialiser produces. On 32-bit ARM those arguments land in different registers. Whether real programs are affected depends on how the compiler lowers the call on this target, which could not be checked without the ARM-enabled compiler.

Division and atomics

Mojo's `Int` is 64 bits, and no ARM core here divides 64-bit numbers in hardware. `aeabi.s` provides restoring division: a 32-iteration core for 32-bit values, and a 64-iteration core for 64-bit ones. It defines both the AEABI names and the compiler-rt spellings, because LLVM does not always emit the AEABI names. The printing path for an `Int` first failed to link on `__moddi3`.

The algorithm is checked by `tools/test_div_model.py`. It mirrors the assembly loops in Python and compares them with native division over 200,000 random pairs, plus ten 64-bit cases. It tests a model of the algorithm, not the assembled code, and its signed 64-bit path is computed but not asserted.

Atomics, per profile

Mojo's strings and lists keep reference counts, which reach the compiler's atomic builtins. Two implementations serve the two profiles:

Profile	Implementation
StrongARM	SWP-based loops. On a single core driven by interrupts, plain loads and stores are atomic enough for the rest
Cortex-A72	LDREX/STREX for add and subtract; dmb for synchronise; LLVM inlines the others on ARMv8

What is missing

Missing	Consequence
a soft-float library	any Float64 fails at link time; fixed point is the idiom
GetArgV	INFERRED: <code>sys.argv</code> should fail to link
the rest of the async runtime	INFERRED: <code>parallelize</code> and <code>async</code> should fail to link
fprintf output	INFERRED: <code>_printf</code> output is silently dropped
threads	none, by design; a Wimp task multitasks co-operatively through <code>Wimp_Poll</code>

The last row is RISC OS, not a gap in the port. The Mandelbrot demo's source says so: its long recompute blocks the whole desktop until it returns (chapter 7).

5. The PRM as data

A RISC OS program does its work through SWIs: numbered calls into the operating system, with arguments in registers. The *Programmer's Reference Manuals* (PRMs) document about six hundred of them. The port's stated objective is that **no end-user program issues a SWI** or names a register. That needs a library covering the whole documented set, and nobody writes six hundred bindings by hand twice — once in C, once in Mojo.

This chapter covers the pipeline that generates both from a database, the register model it applies, the bugs that model had to learn about, and what regenerating the library today would do.

Two layers per SWI

Every bound SWI gets two functions with one contract:

Layer	Where	What it is
a C shim	rostrt/swis_<chunk>.c	one function, named exactly like the SWI, issuing one swi instruction
a Mojo binding	riscos/<chunk>.mojo	a Mojo function calling that C symbol by name

A generated pair looks like this. The C shim:

```
/* Wimp_SetColour (SWI &400E6). See PRM 3-191. */
void Wimp_SetColour(int colour)
{
    register int reg0 __asm("r0") = colour;
    __asm__ volatile("swi 0x400E6" : : "r"(reg0) : "r1", "r2", "r3", "r12", "lr", "memory");
}
```

and, emitted by the same loop in the generator, the binding shape:

```
mojo_lines.append(f"def {fname}({'', '.join(params)}) -> {mret}:")
```

```
mojo_lines.append(f'    ""{name} (SWI &{num:X}).{pr}""')
if retreg is not None:
    mojo_lines.append(
        f'    return external_call["{name}", Int32]({"", ".join(call_args)})'
    )
```

So `wimp.set_colour(7)` in Mojo calls `Wimp_SetColour` in C, which issues SWI `&400E6` with `R0 = 7`. The day the calling mechanism changes, only the generator changes.

From manual to database

The pipeline starts outside the public repositories. A private tool parses the page texts of the PRMs into a SQLite database. **The database is licensed-derived**: the manuals' prose columns are Acorn's text, and the team keeps it private. The generator reads only the database's **factual** columns: SWI names, numbers, which registers are inputs and outputs, and page references.

Database fact	Count
SWI rows	784
documented, with a number	584
index-only, with no contract	200
chunks, the SWI name prefixes	49
service calls	108

Citing, not reproducing

Commit `f75de12` settled how the manual appears in the published bindings: it does not. Generated docstrings carry the SWI's name, number and page reference only. All 93 generated files were scanned for manual text afterwards. The generator's own comment states the rule:

```
# The PRM's own wording is not reproduced here. SWI names, numbers
# and register roles are facts; the manual's prose is Acorn's, and
# this package is published. A citation points a reader at it.
```

Parameter names get the same care. Commit `c9247da` derives them from each register's description with pattern rules (`tools/argnames.py`), and falls back to a positional `arg<N>` when the description does not name a value. That cut register-named parameters from 228 to 21.

The register model

For each documented SWI, the generator decides whether it can bind it, and how. First, the skip rules, in order:

```
if name in HAND_OVERRIDE:
    stats["hand"] += 1
    continue
...
if "reason" in on_entry or "reason code" in on_entry:
    stats["reason"] += 1
    continue
if any(reg_num(e[0]) > 3 for e in entry) or len(entry) > 4:
    stats["toomany"] += 1
    continue
# magic entry constants we cannot express generically
if "'task'" in on_entry or "× 100" in on_entry or "'task'" in on_entry:
    stats["magic"] += 1
    continue
```

The skip rules, explained

Skipped	Why
hand overrides	Wimp_Initialise and Wimp_CloseDown, written by hand (below)
reason-multiplexed SWIs	one number, many behaviours selected by R0, such as the OS_File family
an input above R3, or more than four inputs	a limit of the current shim writer, not of the SWIs
magic entry constants	values the generator cannot express generically

Then, for a bindable SWI:

- **inputs** become C parameters in register order: a pointer if the description mentions a pointer, block, buffer or address, otherwise an `int`
- **outputs** described as *preserved* or *corrupt* are ignored
- **the first real output** becomes the return value, preferring R0; the rest become trailing `int *` out-parameters

Two bugs in the clobber list

The inline assembly names what the SWI may destroy. Both runtime bugs that crashed programs inside the ROM came from that list.

lr, always

The generator adds `lr` to every shim's clobbers, and its comment explains why this is not optional:

```
# "lr" is not optional. SWI banks its return address into r14_svc,
# so code already running in SVC mode – which is every RISC OS module
# entry point – loses its own return address across the SWI. Without
# the clobber the compiler leaves a leaf shim's return address in lr
# and the `mov pc, lr` at the end jumps back into the shim: a one
# instruction infinite loop. Harmless in USR mode, fatal in a module,
```

```
# so declare it always and let leaf shims push lr.
```

In an application, which runs in user mode, a SWI never touches the program's `lr`. In a module, which runs in supervisor mode, the SWI instruction itself overwrites `lr`. A leaf shim that kept its return address there returned to itself, forever.

The 'TASK' word in R3

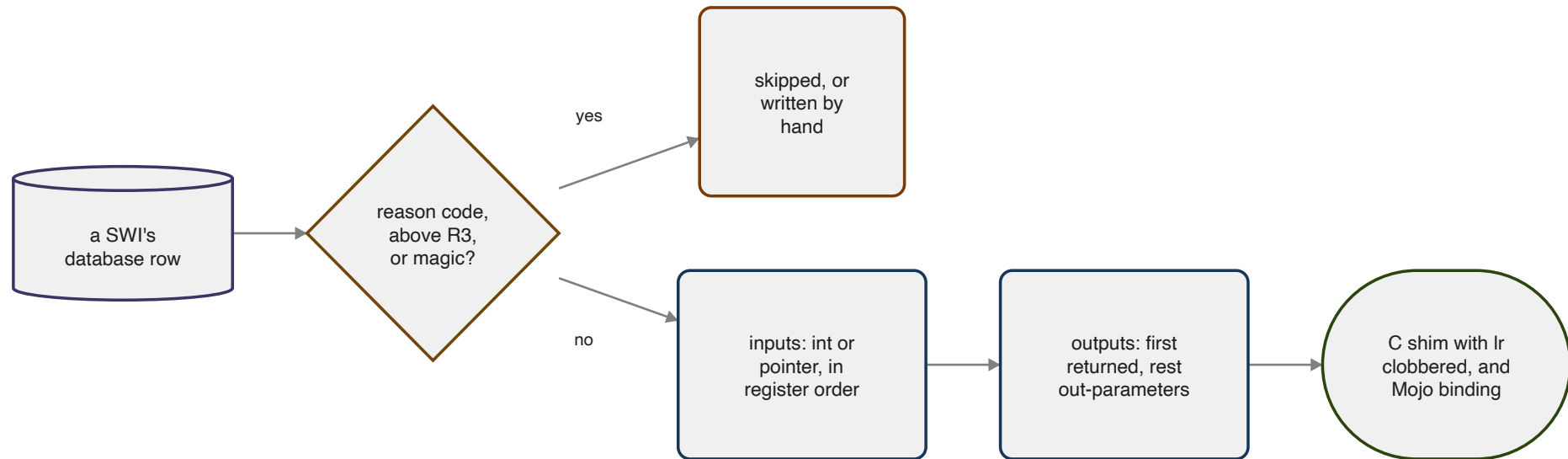
`Wimp_Initialise` is one of the two hand-written shims, because its contract carries a magic constant: `&4B534154`, the characters `TASK`. It also reads R3 as a pointer to a list of messages the task wants. The first version named R3 only in the clobber list, which told the compiler R3 was free scratch space. Commit `b3fe9b9` records what LLVM did with that freedom: it staged the `TASK` constant through R3, and left it there. The Wimp followed `&4B534154` as a message list and took a data abort **inside the ROM**, at `&FC12C9C8`.

The fix makes R3 an explicit input, holding 0, which means every message:

```
int Wimp_Initialise(int version, const char *name, int *out_version)
{
    register int r0 __asm("r0") = version;
    register int r1 __asm("r1") = 0x4B534154; /* 'TASK' */
    register const char *r2 __asm("r2") = name;
    register const int *r3 __asm("r3") = 0; /* all messages */
    __asm__ volatile("swi 0x400C0"
                     : "+r"(r0), "+r"(r1)
                     : "r"(r2), "r"(r3)
                     : "r12", "memory");
    if (out_version)
        *out_version = r0;
    return r1; /* task handle */
}
```

The lesson is written into the source in one line: **a register the SWI reads is an input, never a clobber**. With R3 fixed, the `wimp_window` demo opened its window on RPCemu.

A SWI's registers, classified



Coverage

Commit `c08af21` ran the generator over every chunk, not just `OS` and `Wimp`. Counted by the project's own coverage tool, which imports the generator's rules so the two cannot drift:

	SWIs
bindable by the generator	438
written by hand	2
reachable	440 of 584, 75%
reason-multiplexed, not bound	51
an input above R3, not bound	93

Coverage, continued

Chunk	Reachable of total
OS	77 of 111
Wimp	49 generated + 2 hand of 61
Font	30 of 41
ColourTrans	29 of 36
FileCore	4 of 12

The generated library is 45 modules and 45 shim files: **439 bindings**, covering 438 distinct SWIs. All 45 shim files compile with zero warnings for both profiles. None of them uses a SWI's X form, so an error from any generated call goes to the error handler and stops the program. The unbound SWIs include most file operations and the `OS_SpriteOp` and `OS_Byte` families. So the demos draw with `os.plot` and VDU codes.

The project's own counts do not reconcile: the README says 126, and commits say 449, 451 and 452. The generator's actual output is 439.

Three generator bugs

A wildcard in a prefix. The generator selects each chunk's SWIs with SQL `name LIKE chunk + "_%"`. In SQL, `_` is itself a one-character wildcard. So `Draw_%` matches `DrawFile_DeclareFonts`, which is emitted into `draw.mojo` under the mangled name `ile_declare_fonts`, as well as into its own `DrawFile` module. That is why one SWI is bound twice. **INFERRED:** if both shim objects were ever linked together, `rosc` would silently keep one of them, while a conventional linker would reject the pair.

Rows with no inputs. 82 generated functions take no arguments at all, because their database rows list no entry registers. Some SWIs genuinely take nothing. Many cannot be called usefully without inputs they need.

Wimp_Poll's third register. The generated shim takes three arguments, and puts the third in R3. The hand-written `poll(mask, block)` passes two. **INFERRED:** R3 is then undefined, which is harmless unless the poll-word bit of the mask is set.

Regenerating would undo real fixes

A check regenerated the whole library from the published generator and a copy of the database. **All 45 shim files, and 47 of the 48 Mojo files, came out byte-identical** to the published ones. The generator is faithful.

The one difference is `riscos/wimp.mojo`. The Othello and Mandelbrot work edited it by hand in three commits, and those edits never went into the appendix file the generator appends. So a regeneration would:

- remove `game_window`, `word`, `origin_x` and `origin_y`, `begin_redraw`, `next_rectangle`, `begin_update` and `alloc`
- bring back `PollBlock.words()`, which never compiled
- bring back the dangling out-cell in `initialise`, which crashed Othello

It would break both Wimp demos. Chapter 6 covers the fixes in question. Moving them into `appendix_wimp.mojo` would make the generated output the source of truth again.

The objective, and its admitted compromise, are recorded together in commit `c08af21`. No end-user program issues a SWI. The one-to-one mapping from SWI to function is deliberately *not an abstraction*. And `Int32` for everything pointer- or size-like is acknowledged to be wrong for a future 64-bit RISC OS.

6. A Wimp application: Othello

A program that prints proves the chain works. A desktop application proves the library works: it has to register with the Wimp, create a window, run a poll loop, redraw on request, and turn mouse clicks into actions. This chapter walks through the first two Mojo programs on the Pi 4 — a hello world that names no register, and an Othello game in a window — and through the four library bugs that Othello found.

Hello world, the way the objective says

The first program built and run on the Pi 4, on 12 September (commit `9db30e1167`), is three lines of code:

```
from riscos import os

fn main():
    os.write0("Hello RISC OS, from Mojo.\n")
    os.write0("Built on the host, run on the Pi 4.\n")
```

Its docstring explains why it exists. The older test program reached into the runtime with `external_call["os_write0"]`. That works, and it is exactly what the library exists to stop. This one imports `riscos.os` and names no SWI and no register. The one SWI is in the C code behind `os.write0`, so the day the call mechanism changes, this file does not.

All four farm machines printed both lines.

Othello in a window

`othello_wimp.mojo` (commit `7a3b81b82d`) is the same game as a console demo that plays itself, with a person on one side. Black is the player. White is the console demo's positional engine, which scores corners highly and the squares beside them negatively. **The rules are imported, not rewritten**, so both programs play the same game.

The docstring states the design constraint: *no SWI appears here*. The board is drawn with `os.plot`, and the window comes from `wimp.game_window`.

Published only in part. The game imports its rules from `demos.othello`, and its text helpers from `demos.vdu`. Neither is in the published repository, so a clone cannot build this demo.

The board as geometry

```
comptime SQ = Int32(64)           # OS units down one side of a square
comptime BOARD = SQ * 8
comptime STATUS = Int32(52)       # strip along the top for the score
comptime W = BOARD
comptime H = BOARD + STATUS
```

RISC OS measures the desktop in **OS units**, not pixels, so a 64-unit square has the same size on screen in any resolution. Colours are Wimp palette entries — 0 white, 1 grey, 7 black, 13 dark green for the felt, 10 bright green for hints — given names at the top of the file instead of numbers at each call.

Drawing with PLOT

The drawing primitives are two short functions over the VDU `PLOT` codes:

```
fn fill_rect(x0: Int32, y0: Int32, x1: Int32, y1: Int32):
    """Filled rectangle in the current colour (PLOT 4 move, PLOT 101 fill)."""
    os.plot(4, x0, y0)
    os.plot(101, x1, y1)

fn fill_circle(cx: Int32, cy: Int32, r: Int32):
    """Filled circle: move to the centre, plot to a point on the rim."""
    os.plot(4, cx, cy)
    os.plot(157, cx + r, cy)
```

A window the program redraws itself is given no background, so every pixel is the program's to fill, including the strip behind the score. The score itself is VDU 5 text, drawn at the graphics cursor.

The Wimp layer this needed

Othello needed four things the generated library did not provide. They were written by hand into `riscos/wimp.mojo`, and each was found by needing it.

A window the program draws itself

The library's `simple_window` hands redrawing to the Wimp and fills the work area with icons. That is right for a dialogue and useless for a board. `game_window` builds a `Wimp_CreateWindow` block for a window the program paints:

```
def game_window(title: StringLiteral, width: Int32, height: Int32) -> WindowHandle:
    let block = _arena(Int32(88))

    _w(block, 0)[0] = 300
    _w(block, 1)[0] = 300
    _w(block, 2)[0] = 300 + width
    _w(block, 3)[0] = 300 + height
    _w(block, 4)[0] = 0           # scroll x
    _w(block, 5)[0] = 0           # scroll y
    _w(block, 6)[0] = -1          # open at the top of the stack

    _w(block, 7)[0] = (WF_NEW_STYLE | WF_TITLE_BAR | WF_CLOSE_ICON
                      | WF_BACK_ICON | WF_MOVEABLE)

    ...
    _w(block, 10)[0] = 0
    _w(block, 11)[0] = -height
    _w(block, 12)[0] = width
    _w(block, 13)[0] = 0

    _w(block, 14)[0] = IF_TEXT_INDIRECTED
    _w(block, 15)[0] = BUTTON_CLICK
    ...
    _w(block, 21)[0] = 0          # no icons: we draw everything
```

```
return WindowHandle(create_window(block))
```

Three choices in that block matter:

- **The Wimp-redraws flag is left clear**, so redraw requests come to the program.
- **The work area runs from y = –height to 0**. The origin is at the top left, and y runs negative downwards, which is the Wimp's convention for a document.
- **The work-area button type is set**. That one is silent when wrong.

The source records the last point next to the constant:

```
comptime BUTTON_CLICK = Int32(3) << 12
"""Work-area button type 3, in bits 12-15 of the window's +60 word.

Left at the default 0 the Wimp reports no clicks on the work area at all.
The events simply never arrive, which reads as a broken poll loop rather
than as a window that asked not to be told.
"""
```

The window block lives in the runtime's never-freed arena, because the Wimp keeps pointers into it, such as the indirected title, for the window's whole life.

Where the work area is on screen

Redraw requests, open requests and `Wimp_UpdateWindow` all leave the window's visible area and scroll offsets in the same words of the block. So one piece of arithmetic gives the screen position of the work-area origin:

```
def origin_x(block: PollBlock) -> Int32:
    return word(block, 1) - word(block, 5)

def origin_y(block: PollBlock) -> Int32:
```

```
"""Screen y of the work-area origin: visible y1 minus scroll y."""
return word(block, 4) - word(block, 6)
```

Drawing now, not later

To repaint after a move, a program can ask the Wimp to redraw later, with `Wimp_ForceRedraw`, or draw immediately, with `Wimp_UpdateWindow`. `Wimp_ForceRedraw` has more than four input registers, so the generator never bound it. `begin_update` wraps the other call, which is also the better one. It draws now, instead of asking to be asked later:

```
fn repaint(win: wimp.WindowHandle, block: wimp.PollBlock,
          b: UnsafePointer[UInt8, MutUntrackedOrigin],
          turn: Int32, over: Bool):
    """Draw now, rather than asking the Wimp to ask us later."""
    var more = wimp.begin_update(win, block, 0, -H, W, 0)
    while more != 0:
        draw(b, wimp.origin_x(block), wimp.origin_y(block), turn, over)
        more = wimp.next_rectangle(block)
```

The loop is the standard Wimp redraw loop. The Wimp hands out one visible rectangle at a time, clipped, until there are none left.

A method that had never compiled

`PollBlock.words()` views the 256-byte poll block as 32-bit words. It had never compiled: it built an `Int32` pointer from a `UInt8` one, which does not typecheck. Nothing had called it, and the compiler does not elaborate an uncalled method body, so it sat in the library looking correct. It now uses `bitcast`:

```
fn words(mut self) -> UnsafePointer[Int32, MutUntrackedOrigin]:
    return self.ptr.bitcast[Int32]()
```

The poll loop

With those pieces, the main loop reads like any RISC OS application. The board is 64 bytes on the stack:

```
while running:
    let event = wimp.poll(1, block)      # mask 1: no null events wanted

    if event == wimp.REDRAW_WINDOW_REQUEST:
        var more = wimp.begin_redraw(block)
        ox = wimp.origin_x(block)
        oy = wimp.origin_y(block)
        while more != 0:
            draw(b, ox, oy, turn, passes >= 2)
            more = wimp.next_rectangle(block)

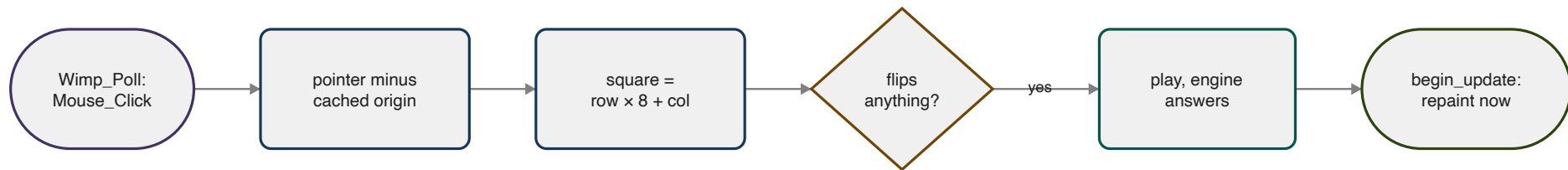
    elif event == wimp.OPEN_WINDOW_REQUEST:
        wimp.open_window_from_poll(block)
        ox = wimp.origin_x(block)
        oy = wimp.origin_y(block)

    elif event == wimp.CLOSE_WINDOW_REQUEST:
        running = False

    elif event == wimp.MOUSE_CLICK:
        let dx = wimp.word(block, 0) - ox
        let dy = oy - wimp.word(block, 1) - STATUS
        if dx >= 0 and dx < BOARD and dy >= 0 and dy < BOARD:
            let sq = (dy // SQ) * 8 + (dx // SQ)
            if turn == BLACK and passes < 2 and flips(b, sq, BLACK) > 0:
                play(b, sq, BLACK)
            ...
```

From a click to a move

The click handler relies on a detail recorded in its comment. **A mouse-click block carries the pointer's position, not the window's.** So the origin used is the one cached from the last redraw or open event. Every window move raises an open request, so the cache stays current. Illegal squares do nothing, because `flips` returns zero for them.



A bug that hid until an unrelated edit

A one-line change to the drawing code made Othello die at start-up with *Internal error: abort on data transfer at &000086DC*. The faulting instruction was a store just after `Wimp_Initialise` returned, in `main`, which had not been touched. Removing the drawing change made it go away. Commit `805c4551eb` recognised this as the shape of a latent memory bug rather than a new one.

`initialise` had taken its version out-cell from a helper:

```
def _version_out() -> UnsafePointer[Int32, MutUntrackedOrigin]:
    """Scratch cell for the Wimp version out-value."""
    return stack_allocation[1, DType.int32, 4]()
```

That returns a pointer into the helper's own stack frame, which is dead as soon as the helper returns. The stack just below `initialise` is exactly where the `Wimp_Initialise` shim's frame then lands. The SWI wrote its version number through the stale pointer, over the registers saved in the shim's live

frame. `initialise` returned with a damaged stack pointer, and the next store aborted. Whether that was fatal depended on inlining and frame sizes, which is why it hid.

The fix allocates the cell in `initialise` itself, whose frame is live across the call:

```
def initialise(name: StringLiteral) -> TaskHandle:
    var version_out = stack_allocation[1, DType.int32, 4]()
    let task = external_call["Wimp_Initialise", Int32](
        Int32(310), name.ptr(), version_out
    )
    return TaskHandle(task)
```

The commit adds that anything else returning `stack_allocation` from a helper would have the same bug, and that this was the only one. The fix went into `wimp.mojo`, but not into the appendix the generator reads, so regenerating the library would bring the bug back (chapter 5).

Polish, from watching it run

Commit `6e23dc2431` made two changes from watching the game played:

- **The score sat on the board.** VDU 5 text hangs *below* the point it is plotted at, rather than sitting above a baseline. So the y coordinate is the top of the text. A value of `-34` in the 52-unit strip put the bottom of every glyph on the green; `-12` centres a 32-unit character cell in the strip.
- **Legal moves are shown**, as small green dots, drawn after the discs. `flips` is zero for an occupied square, so a dot can never appear under a piece.

Verified on the Pi 4

Recorded on farm machine alpha:

Check	Result
click c4	plays it, and flips d4
the engine's reply	d3
the score	Black 3, White 3
illegal squares	ignored
the close icon	closes the game
hints in the opening position	d3, c4, f5 and e6, the four legal openings, and no others

7. Fixed point on the desktop: Mandelbrot

The second Wimp demo draws the Mandelbrot set in colour, in a window, and zooms when clicked. It is small, and every design choice in it answers a real constraint of this platform: no floating point, a co-operatively multitasked desktop, a slow redraw path, and a link line that names its objects by hand. This chapter walks through those choices.

Q16.16, because there is no floating point

Chapter 2 removed the floating-point registers, and chapter 4 has no soft-float library, so a `Float64` anywhere fails to link. The demo's docstring does not treat fixed point as a workaround: *it is what you would have written for a StrongARM in 1996*, and far quicker than any soft-float emulation.

In **Q16.16** a 32-bit integer holds a number times 65,536: sixteen bits of whole part and sixteen of fraction.

```
comptime ONE = 65536           # 1.0 in Q16.16
comptime FOUR = 4 * ONE       # escape radius, squared
```

The only non-trivial operation is multiplication. The product of two Q16.16 numbers needs 64 bits on the way through, before shifting back:

```
@always_inline
fn qmul(a: Int32, b: Int32) -> Int32:
    """Q16.16 multiply: the product needs 64 bits on the way through.

    On a 32-bit ARM that is a register pair, and LLVM turns it into a
    single SMULL - an instruction even a StrongARM has.
    """
    return Int32((Int64(a) * Int64(b)) >> 16)
```

`SMULL` multiplies two 32-bit values into a 64-bit result in one instruction. So the inner loop costs three multiplies per iteration, each a single instruction.

Constants as integer arithmetic

One compile error shaped the source. `Int32(-0.75 * ONE)` does not compile, because a float literal will not convert. So the starting view is written as exact integer arithmetic:

```
var centre_x = Int32(-3 * ONE // 4)
var centre_y = Int32(0)
var half = Int32(3 * ONE // 2)
```

The commit's comment: in a fixed-point program, the integer form is the honest one anyway.

The escape loop

The view is a centre and a half-span rather than a pair of corners, because that is what zooming acts on. Each cell iterates $z \leftarrow z^2 + c$ until $|z|^2$ passes 4:

```
var cy = upper
for row in range(ROWS):
    var cx = left
    for col in range(COLS):
        var zr: Int32 = 0
        var zi: Int32 = 0
        var n: Int32 = 0
        while n < max_iter:
            let zr2 = qmul(zr, zr)
            let zi2 = qmul(zi, zi)
            if zr2 + zi2 > FOUR:
                break
            let t = zr2 - zi2 + cx
            zi = qmul(zr, zi) * 2 + cy
            zr = t
            n += 1
        buf[row * COLS + col] = UInt8(colour_of(n, max_iter))
```

```
    cx += step
    cy -= step
```

Rows run down the screen while the imaginary axis runs up, so the loop starts at the top of the range and subtracts. The escape count becomes one of eight Wimp colours, cycling; points inside the set are black. The docstring explains the cycling: a steady ramp over only sixteen desktop colours would put most of the interesting structure in one shade.

The grid is 128×128 cells of 4 OS units each. The vertical range is ± 1.5 , where the ASCII demo uses ± 1.25 . That is not a different view, commit `c4b9c847dd` explains. Character cells are twice as tall as they are wide, and these cells are square.

Three design decisions

Commit `c4b9c847dd` names three things in the program as *design rather than detail*.

1. Compute before becoming a task

```
fn main():
    # Before Wimp_Initialise on purpose: this takes a moment, and a task
    # that is not yet a task cannot hold up anybody else's redraw.
    let buf = wimp.alloc(Int32(COLS * ROWS))
    ...
    compute(buf, centre_x, centre_y, half, max_iter)

    let task = wimp.initialise("Mandelbrot")
```

RISC OS multitasks co-operatively. While a Wimp task is computing and not calling `Wimp_Poll`, no other application on the desktop runs. A program that is not yet a task holds nobody up. Computing inside the poll loop would freeze the desktop for the duration; computing inside a redraw would repeat that on every expose.

2. Keep the picture, not just the maths

The result is kept as one byte per cell, so a redraw repaints instead of recomputing. The buffer comes from `wimp.alloc`, which exposes the runtime's never-freed arena. In the commit's words, a program that draws needs somewhere to keep what it drew.

3. Run-length encode each row

Drawing is the slow path: every rectangle is a `PLOT` call through the VDU drivers. The set has long bands of one colour, so each row is drawn as runs:

```
for row in range(ROWS):
    let y1 = plot_top - Int32(row) * CELL
    let y0 = y1 - CELL
    var col = 0
    while col < COLS:
        let c = Int32(buf[row * COLS + col])
        var run = 1
        while col + run < COLS and Int32(buf[row * COLS + col + run]) == c:
            run += 1
        wimp.set_colour(c)
        fill_rect(ox + Int32(col) * CELL, y0,
                  ox + Int32(col + run) * CELL - 1, y1 - 1)
        col += run
```

A band of one colour costs one rectangle instead of one per cell.

Zoom, and its floor

Select zooms in on the clicked point, and Adjust zooms back out. That is the RISC OS convention for a pair of opposite actions, and it saves inventing a menu for two verbs (commit `3ce3feed0d`):

```
if buttons == SELECT:
    let next_half = half // 2
```

```
if step_for(next_half) >= MIN_STEP:
    let step = step_for(half)
    centre_x = centre_x - half + (dx // CELL) * step
    centre_y = centre_y + half - (dy // CELL) * step
    half = next_half
    depth += 1
    changed = True
elif buttons == ADJUST:
    if depth > 0:
        half = half * 2
        depth -= 1
        changed = True
```

Iterations rise with depth: 64 at the top, then 32 more per zoom, capped at 256. A deeper view at a fixed 64 iterations reads as almost entirely inside the set.

Q16.16 sets the floor

The fraction is sixteen bits, so the finest step is 1/65,536. From a span of 3.0 across 128 cells, the step between cells starts at 1,536 units and halves with each zoom:

Zoom	Step between cells, in 1/65,536
0	1,536
3	192
6	24
9	3

By the ninth zoom the picture is visibly quantised. So zooming in is **refused** once the next step would fall below 2, instead of letting the picture dissolve into blocks. The docstring is precise about the remedy: more iterations do not help, and going deeper needs a wider fixed point.

One click, end to end



No hourglass, and why

The recompute after a zoom blocks `Wimp_Poll`, and so blocks the whole desktop. The RISC OS way of admitting that is an hourglass pointer, and the source says one belongs here. It cannot have one yet:

```

# An hourglass belongs here and cannot have one yet:
# hourglass.on() exists as a binding and as a C shim, but
# only swis_os and swis_wimp of the library's 45 modules
# are ever compiled and linked, so it fails with
# "undefined: Hourglass_On". The fix is the library
# becoming an archive the linker draws from, not another
# object hardcoded into the link line.

```

This is the link line from chapter 1 meeting the linker limit from chapter 3. **INFERRED:** `rosc` reads archives but includes every member, so an archive alone would link all 45 shim modules into every program — including both copies of the duplicated `DrawFile` function from chapter 5. The fix the comment asks for also needs **archive member pulling** in `rosc`: including only the members that resolve undefined symbols, which is already on its list of gaps.

The comment also records the real answer if the recompute ever takes longer than a second or so: slice the work across Wimp null events, so the desktop keeps running between rows.

Verified on the Pi 4

Recorded on farm machine charlie:

Action	Result
Select on the neck between the cardioid and the period-2 bulb	zoom 2, at 96 iterations, centred there
Adjust	back to zoom 1, at 64 iterations
the zoom's recompute	runs through the arena in the application slot, after commit d2ab069

8. Running and verifying it

A toolchain that produces images proves nothing until something runs them and checks what happened. This chapter covers how Mojo programs reached the emulated Pi 4 and were checked there: the build script, the host share, the four-machine farm, a typing tool that had to slow down, and self-checking programs. It ends with the StrongARM sandbox on RPCEmu, where the earliest bugs were caught, and a ledger of what ran where.

pi4.sh: build, link, deliver

The published build script takes program names and does the whole host side:

```
riscos-test/pi4.sh hello_world           build only
riscos-test/pi4.sh --to alpha hello_world build, then into alpha's share
riscos-test/pi4.sh --to alpha --as hello hello_world
```

For each name, it compiles and links with the commands in chapter 1, then copies the image into the named farm machine's share. Two behaviours came from use:

- **A failing program does not end the sweep** (commit `d41626b038`). The point of building them all is to find out how many are bad.
- **The image is written beside its source**, as `name_a72, ff8`, so the Pi 4 build and the StrongARM build of the same demo do not overwrite each other.

A failed compile prints the first three error lines from the compiler's log. A successful link prints the image size, and where it was delivered.

Getting the file into the guest

Each farm machine has a HostFS share: a directory on the host that the guest sees as a filing system. The [HostFS walkthrough](#) covers how that works. The build script's comment records what was measured on the farm, rather than assumed, about getting a program to run from it.

The HostFS on the farm's card image at that time **stored no file metadata**. `*Ex` showed load and exec as zero, and `*SetType` was accepted and silently discarded. But `*Run` honours the `, ff8` suffix as the filetype. So the whole guest side is one command in a task window:

```
*Run HostFS:$.hello_world,ff8
```

Two guesses about what else was needed were both wrong, and the commit says so. The image did not need copying to the card first. And there was no ten-character name limit to work around.

The farm

`riscos-pi4/tools/farm.py`, in the QEMU fork, runs **four named RISC OS machines that cannot tread on each other**. Its docstring names the three things two copies of the emulator would otherwise share, and each is a way for them to corrupt each other's work:

Shared by default	What each farm machine gets instead
the disc	its own qcow2 overlay over one read-only card image; a new machine costs a few hundred kilobytes, not two gigabytes
the HostFS share	its own directory, so build outputs cannot collide
the QMP control port	its own port, 4471 to 4474; the single-machine launcher's 4461 is left alone

The machines are named alpha, bravo, charlie and delta. The docstring's reason for names: "charlie is wedged" is a sentence, while "instance 2 is wedged" is a lookup. They run headless by default. QMP does screendumps and input perfectly well with no display, and four windows is not something anyone wants.

The farm's commands

Command	Does
<code>farm.py create</code>	makes, or repairs, all four instances
<code>farm.py up alpha, up all</code>	starts one, or all
<code>farm.py status</code>	which are running, on which port
<code>farm.py shot alpha</code>	a screendump into that machine's screen directory

The farm, continued

Command	Does
<code>farm.py hmp alpha "info registers"</code>	a QEMU monitor command
<code>farm.py down bravo</code>	stops one, politely and then firmly
<code>farm.py reset charlie</code>	throws away its disc writes and keeps its share
<code>farm.py ls</code>	names, ports and paths

Typing at a machine

`keys.py` types command lines into a farm machine over QMP, and reads the screen back:

```
keys.py alpha --f12 --line "Cat HostFS:" --shot
keys.py alpha --line "Run HostFS:$.hello" --shot --wait 4
```

Its docstring records three decisions:

- **Explicit key-down and key-up events, never QMP's send-key.** `send-key` goes through a delayed queue, which interleaves with immediate events and scrambles the shift state halfway through a word.
- **The guest keymap is UK.** The shifted punctuation is not the US arrangement: `"` is Shift-2, and `@` is Shift-apostrophe.
- **Only the characters a command line needs are mapped.** Anything else raises an error, rather than silently typing something different.

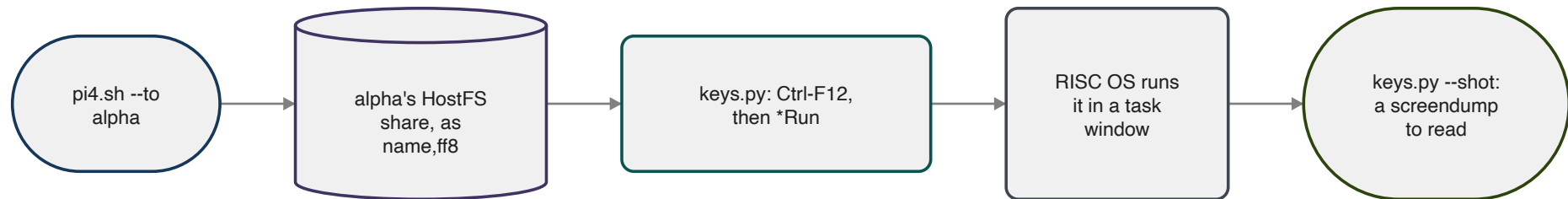
Why typing got slower

The first timings held each key for 30 ms. Commit `ac5223853d` doubled them after `$.othello,ff8` arrived at the guest as `$>0HELL<FF*`, twice, on two different machines. It was not a mapping error. The guest polls the emulated USB keyboard on its own schedule, and with four machines running, key transitions that fell between two polls were never seen. A missing *shift release* is the expensive case. The rest of the word arrives shifted, and the line becomes a plausible-looking command that fails for a reason that has nothing to do with typing. So the investigation goes to the filing system instead.

```
HOLD = 0.06      # how long a key stays down
GAP = 0.06       # and the pause before the next one
MODGAP = 0.03    # settle after pressing a modifier, and before releasing
```

The commit ends on the point: *typing a command line is not a benchmark*.

One program, from host to screen



Programs that check themselves

A program that merely starts proves little. Commit `d41626b038` built all six console demos for the Pi 4, and ran the two that **check their own answers**:

Program	Prints	Expected
fib	fib(90) = 2880067194370816120	2880067194370816120
sieve	pi(10000) = 1229	1229, largest prime 9973

`fib(90)` does not fit in 32 bits, so that line also proves the runtime's 64-bit division and arithmetic working on the A72. Both answers were checked again after the heap moved into the application slot.

The same commit explains how the NEON fault in chapter 2 went unnoticed for so long. The demo set had only ever been built for the StrongARM, which has no NEON for LLVM to reach for. So the StrongARM builds got the right answer, for a reason that did not hold on a Cortex-A72.

The StrongARM sandbox

Before the Pi 4 farm existed, programs ran on **RPCEmu Instrument**, a fork of RPCEmu with a headless interpreter driven by JSON-RPC:

Tool	What it does
the fault trap	stops on the faulting instruction, and reads symbols from the ELF the toolchain writes beside its image
breakpoints and watchpoints	by address or by symbol
snapshots	a booted machine restored in 0.020 s from 1.5 MB, against a 9-second boot
the guest portal	a 504-byte RISC OS module that lets a host script drive the guest

That sandbox caught the earliest runtime bugs. The fault trap found the missing stack in chapter 4, with `r13 = &80000000`. The `Wimp_Initialise` register bug in chapter 5 aborted inside the ROM there, and after the fix the `wimp_window` demo opened its window. The Mojo README calls the tool a GDB stub. It is not one: it is the JSON-RPC fault trap.

What ran, where

Date	Where	What	Evidence
8 Sep	build only	<code>hellomojo</code> , ff8, 432 bytes	disassembly
9 Sep	RPCEmu, StrongARM	<code>wimp_window</code> opened a window	ROSCC b3fe9b9
9 Sep	RPCEmu, StrongARM	<code>natural</code> , ff8 printed through print	the review of 10 Sep
10 Sep	RPCEmu, StrongARM	a Mojo relocatable module: 48 of 48 live checks	the review of 10 Sep

What ran, where, continued

Date	Where	What	Evidence
12 Sep	Pi 4 farm, all four machines	hello_world printed both lines	9db30e1167
12 Sep	Pi 4 farm	fib and sieve checked their own answers	d41626b038
12 Sep	Pi 4, alpha	Othello played by mouse	7a3b81b82d, 6e23dc2431
12 Sep	Pi 4, charlie	Mandelbrot zoomed in and out	3ce3feed0d
12 Sep	Pi 4 farm	all of them again, after the heap moved to the slot	ROSCC d2ab069

Every entry is recorded by the project, in commit messages or its review. Screens were read with screendumps, and the records note one rule of evidence: read the screendump just taken.

Not recorded anywhere: compile times, run times or benchmarks for any Mojo program on either machine.

9. The unpublished half

This last chapter collects what the published repositories do not contain, what that means for anyone trying to reproduce the work, and an honest ledger: the project's stated gates against what later commits show, the risks found by reading the code, and the licensing points a reader should know.

Relocatable modules

RISC OS extends itself with **relocatable modules**: code loaded into the module area, the RMA, and entered in supervisor mode through a header of entry points. HostFS and GVFill are modules. A language that can build modules can extend the OS itself.

Advertised, but not published

The ROSCC README lists a second link mode, `rosc link --module`, linking at base 0 into a relocatable module with its header taken from a `.module` section. **The published source has no such option.** Given `--module`, the published linker treats it as a file name.

rosc's own review of 10 September names the files the module path lives in — `src/module.rs`, a module runtime, a header generator, a build script, a live test, and tests — and its first item says they are **untracked**. It calls the module path the best work in the crate, and asks for it to be committed before anything touches it. The review also mentions a debugging switch that is absent from the published source. **INFERRED:** the rosc used for the demos and for the first HostFS module was newer than the published source.

The module ABI, reconstructed

The QEMU fork's first HostFS module was built with rosc on 10 September, and that build script is public. With the review, it shows the module ABI:

```
# -O0 and -mno-movt matter: they force literal-pool addressing of
# statics (SBREL32), the one static-base relocation the module linker
# applies. MOVW/MOVT pairs (BREL) come out at -O1, or on any CPU with
# Thumb-2 extensions unless movt is disabled
CC_FLAGS="--target=$TRIPLE -mcpu=$CPU -mfloat-abi=soft -ffreestanding -nostdlib -fno-builtin -fropi -frwpi -O0 -mno-movt $MOD_ARENAS"
...
```

```
"$ROSCC" link --module -o "$OUT/HostFS,ffa" \
"$OUT/module_head.o" "$OUT/entries.o" "$OUT/hostfs.o" \
"$OUT/rostrt.o" "$OUT/swis_os.o" "$OUT/modrt.o" \
"$OUT/aeabi.o" "$OUT/atomics.o"
```

The pieces of the ABI

Piece	How it works
code	-fropi: position-independent, reached PC-relative
statics	-frwpi: reached relative to a static base in r9, and claimed from the RMA at initialisation
addressing	-O0 -mno-movt, so statics use literal pools: the one static-base relocation the module linker applies
header and veneers	generated from a title, help text, init and final functions, and commands
runtime	built with -DROSTRT_STATIC_HEAP, because a module has no application slot

The entry veneers are hand-written assembly in the same fork. Each one saves the caller's registers as a block, loads the static base from the module's private word, calls C, and turns a non-zero return into an error with the V flag set:

```
hostfs_fs_\name:
    stmfd    sp!, {r0-r9, lr}      @ the caller's registers, as a block
    mov      r0, sp                @ -> that block
    ldr      r9, [r12]             @ static base from the private word
    ldr      r1, [r9, #-8]         @ the module workspace pointer
    bl       \csym
    cmp      r0, #0
    bne      91f
    ldmfd    sp!, {r0-r9, lr}      @ hand back whatever the handler wrote
    msr      cpsr_f, #0
    mov      pc, lr
91: mov      r11, r0               @ park the error pointer out of the
```

```
ldmfd    sp!, {r0-r9, lr}      @  unwind's reach
mov      r0, r11
msr      cpsr_f, #(1 << 28)
mov      pc, lr
```

Two runtime rules exist only for modules, and both appear earlier in this document. Every SWI shim clobbers `lr`, because a SWI in supervisor mode overwrites it (chapter 5). And `GetOrCreateGlobal` must return the same block on every entry (chapter 4).

Where modules stand

Build	State
a Mojo module, StrongARM	48 of 48 live checks passed on RPCEmu, cold boot 8 s, driven through the guest portal
the same module, Pi 4	does not link: with <code>-fropi -frwpi</code> for ARMv8, clang reaches statics through MOVW/MOVT pairs, relocation types 84, 85, 45 and 46, which the linker refuses
HostFS, built with rosc for the Pi 4	built on 10 September with the <code>-O0 -mno-movt</code> workaround; HostFS later moved to a DDE build to run from ROM

The review's first fix for the Pi 4 is to add the four relocation types, with the same immediate packing as the existing MOVW/MOVT cases.

INFERRED, from the public build script: rebuilt today, it would pick up the runtime's application-slot heap, because it does not pass `-DROSTRT_STATIC_HEAP`. And it assembles the SWP atomics for the Cortex-A72, where ARMv8 no longer provides SWP in AArch32.

What a clean clone can do

Wanted	From public sources today
build rosc	yes, after pointing its LLVM configuration at a local install
build the runtime objects	yes: all 45 shim files compile with zero warnings, for both profiles
regenerate the bindings	no : the SWI database is private
build the ARM-enabled mojo	no, as published : the backend list lacks ARM; add it and rebuild, about an hour

What a clean clone can do, continued

Wanted	From public sources today
build hello_world	only with a locally ARM-enabled mojo
build Othello or Mandelbrot	no : they import the unpublished demos/
link a relocatable module	no : the module writer is unpublished

Gates, stated and actual

The fork's README has not changed since 11 September. Later commits have moved past it:

Gate	README says	Later commits show
R1: the ARM backend in LLVM	done	done, in a local build; not in the published configuration
R2: a RISC OS target in the driver; rosccl as linker	open	still open; the link is a separate step
R3: the standard library's system layer on SWIs	first cut	the runtime surface of chapter 4

Gates, continued

Gate	README says	Later commits show
R4: the Wimp in the library, generated from the PRM	done for 126 SWIs	440 of 584 SWIs reachable
R5: a Wimp window in Mojo, running	built, not yet run	windows run on RPCEmu, and on the Pi 4

The README and the port log also still describe VFPv4 and NEON for the A72 profile, and call RPCEmu's fault trap a GDB stub.

Risks found by reading the code

Each item is **INFERRED**, and not tested:

Where	Risk
KGEN_CompilerRT_GetOrCreateGlobal	rostrt's signature differs from the upstream runtime's; on 32-bit ARM the arguments land in different registers (chapter 4); check with the real compiler
crt0 and os_exit	an exit code is passed without the word OS_Exit needs before honouring it
wimp.report_error	copies the message from byte 1 of the error block, where RISC OS error text follows a 4-byte number
wimp.simple_window	never sets the horizontal scroll word, and the arena is no longer zeroed .bss

Risks found by reading the code, continued

Where	Risk
wimp.poll	the generated shim takes a third register the hand binding never passes
the linker	locals binding to same-named globals; GOT slot collisions; ignored MOVW/MOVT addends
.gitignore	its patterns miss the build products pi4.sh writes under build/ and demos/
long strings	whether the alignment fix cured them is recorded both ways (chapter 4)

Licensing, in brief

Item	Position
ROSCC	MIT since 707a35f; Apache-2.0 before. The commit notes that Apache 2.0 carries an express patent grant and MIT does not. Its NOTICE records that LLVM is linked, not distributed, and that no RISC OS Open or Acorn source is reproduced
MojoRISCOS	Apache-2.0 with LLVM exceptions, inherited from upstream. Modular's per-file headers must be kept; no Modular binaries; nothing routed to Modular for support
names	Mojo, MAX and Modular are Modular's trademarks. The fork is not Modular's, and a build must not be presented as if it were. This document's title names the language the fork compiles, not an official product

Licensing, continued

Item	Position
max/	back in the published tree with the full history, and out of scope: the README gives a licensing reason. Its wording and the NOTICE's differ on which licence governs it
the SWI database	derived from the PRMs, and kept private. Generated bindings cite page numbers, and quote nothing
the QEMU fork and RPCEmu Instrument	GPL-2.0-or-later; the short excerpts in this document are quoted with attribution

The review's order of work

rosc's review of 10 September ends with a priority order. It is still the shortest honest description of what comes next:

1. commit the module work
2. add the `MOVW/MOVT` static-base and PC-relative relocations, so the Pi 4 module links

3. fix section alignment and duplicate-symbol detection
4. build a runner for the QEMU Pi 4 once HostFS can carry files — which it now can, and which the farm and `keys.py` already do by hand
5. generate SWI X-form variants, and handle UTF-8 on a Latin-1 console
6. tooling hygiene

To that list, this document would add the items the demos found: move the hand-written Wimp fixes into the generator's appendix, so regeneration keeps them; publish `demos/` ; and give rosc archive member pulling, so the library can become the archive the Mandelbrot demo's comment asks for.